⚡ PDF Manager

Cursor Implementation Prompts

Module-by-Module • E2E Tests at Every Step • Self-Healing on Failure

Total Modules	9 (01–09)
E2E Test Cases	79 total across all modules
Self-Healing	Every module auto-fixes failures
Backend	FastAPI + Celery + PostgreSQL + Redis
Frontend	React + TypeScript + Vite + Tailwind

How to Use These Prompts

Each module below contains one or more prompts to paste directly into Cursor's chat or Composer. **Copy the entire prompt box verbatim.** Cursor will generate code, run tests, and self-heal any failures before you proceed to the next module.

Self-Healing Rule (applies to every module)

Every prompt ends with a Self-Healing Rule. This tells Cursor: if any test fails → read the error → locate the bug in source code (not the test) → fix it → re-run the failed tests → repeat until all pass. Do not move to the next module until 100% of tests in the current module pass.

Execution Order

Follow modules in order: 01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09. Each module assumes the previous ones are fully implemented and tested.

Test Philosophy

Tests are E2E (end-to-end) — they hit the real running API via httpx, use the real database, and exercise the real Celery workers. No mocking of business logic. Tests are independent and clean up after themselves.

Module Overview

Module	Title	Key Deliverables	Tests
01	**Project Scaffold & Infrastructure**	Docker Compose, project structure, environment config, database, Redis, logging foundation	Included
02	**Authentication Module**	Sign up, login, logout, JWT access + refresh tokens, password hashing, token blocklist in Redis	Included
03	**PDF Upload Module**	Multi-file upload, async Celery processing, metadata extraction, status polling	Included
04	**PDF Viewing & File Management**	Dashboard listing, streaming viewer, rename, soft-delete with 30-day purge	Included
05	**PDF Editing Module**	Server-side PDF editing with pikepdf: insert text, erase regions, save versions	Included
06	**User Profile Module**	View profile, update display name, change email with password confirmation, change password	Included
07	**Live Log Viewer Module**	SSE real-time log stream to browser, filterable log history API, React log terminal	Included
08	**React Frontend — Full UI**	All pages: Auth, Dashboard, Viewer, Editor, Profile, Log Terminal — wired to the API	Manual
09	**Integration & Final E2E Suite**	Full cross-module E2E scenarios, CI smoke test script, coverage report	5 Journeys

MODULE 01

Project Scaffold & Infrastructure

Docker Compose, project structure, environment config, database, Redis, logging foundation

STEP 1 OF 1 Project Scaffold Prompt

```
You are setting up a full-stack PDF Manager application from scratch.
Create the following complete project structure and all configuration files:

DIRECTORY STRUCTURE:
pdf-manager/
  backend/
    app/
      api/          # FastAPI routers
      core/         # config, security, logging
      db/           # SQLAlchemy models, session, migrations
      services/     # business logic
      tasks/        # Celery tasks
      schemas/      # Pydantic models
    tests/
      e2e/
    alembic/
    Dockerfile
    requirements.txt
  frontend/
    src/
      components/
      pages/
      hooks/
      api/          # axios client
    Dockerfile
  docker-compose.yml
  nginx/nginx.conf
  .env.example

DOCKER COMPOSE must define these services:
  - postgres (PostgreSQL 16, persistent volume, healthcheck)
  - redis (Redis 7, persistent volume, healthcheck)
  - api (FastAPI via uvicorn, depends_on postgres + redis, mounts ./backend)
  - worker (Celery worker, same image as api, entrypoint: celery -A app.tasks
worker)
  - beat (Celery beat, entrypoint: celery -A app.tasks beat)
  - frontend (React/Vite, depends_on api)
  - nginx (reverse proxy, port 80, routes /api -> api:8000, / -> frontend:5173)

BACKEND requirements.txt must include:
  fastapi, uvicorn[standard], sqlalchemy[asyncio], alembic, asyncpg,
  celery[redis], redis, python-jose[cryptography], passlib[bcrypt],
  python-multipart, aiofiles, pikepdf, slowapi, pydantic-settings,
  pytest, pytest-asyncio, httpx, pytest-cov
```

```
FRONTEND: initialise with Vite + React + TypeScript.
  Install: axios, react-router-dom, @react-pdf-viewer/core, pdfjs-dist,
  tailwindcss, lucide-react, react-dropzone, react-hot-toast

LOGGING FOUNDATION - create backend/app/core/logging.py:
  - Structured JSON logger using Python's logging module
  - Every log entry has: timestamp, level, module, event, user_id,
    request_id, task_id, duration_ms, metadata (dict), error (optional)
  - DatabaseLogHandler class that writes log rows to a 'logs' DB table
    using a background thread queue (never blocks the request thread)
  - get_logger(module_name) factory function used across the entire app
  - log_event(logger, event, user_id=None, request_id=None, **meta) helper

REQUEST ID MIDDLEWARE - create backend/app/core/middleware.py:
  - FastAPI middleware that generates a UUID request_id on every request
  - Stores it in a ContextVar so all log calls in that request auto-attach it
  - Adds X-Request-ID header to every response

DATABASE MODELS - create all SQLAlchemy async models:
  users: id(UUID PK), email(unique), display_name, hashed_password,
         created_at, updated_at
  pdfs:  id(UUID PK), owner_id(FK users), name, storage_path, size_bytes,
         page_count, status(enum: pending|ready|error|deleted), version(int),
         created_at, updated_at, deleted_at
  pdf_versions: id, pdf_id(FK), version(int), storage_path, saved_at, saved_by(FK
users)
  logs:  id(UUID PK), timestamp, level, module, event, user_id, request_id,
         task_id, duration_ms, metadata(JSONB), error(text)

Run Alembic init and create the initial migration for all models.
The api service entrypoint must run 'alembic upgrade head' before starting
uvicorn.

Create .env.example with all required variables:
  DATABASE_URL, REDIS_URL, SECRET_KEY, ALGORITHM=HS256,
  ACCESS_TOKEN_EXPIRE_MINUTES=15, REFRESH_TOKEN_EXPIRE_DAYS=7,
  FILE_STORAGE_PATH=./storage, MAX_UPLOAD_SIZE_MB=50

E2E TEST - after scaffolding, create tests/e2e/test_00_infrastructure.py:
  - test_postgres_reachable: connect to DB, run SELECT 1
  - test_redis_reachable: ping Redis
  - test_all_tables_exist: verify users, pdfs, pdf_versions, logs tables exist
  - test_api_health_endpoint: GET /api/health returns {status: ok}
  - test_logging_writes_to_db: call log_event(), verify row appears in logs table
  - test_request_id_header: any GET /api/health response has X-Request-ID header

Run all tests. If any test fails, diagnose the failure, fix the root cause in the
source code (not the test), and re-run until all 6 tests pass.
Print a final summary: PASSED X/6 tests.
```

MODULE 02

Authentication Module

Sign up, login, logout, JWT access + refresh tokens, password hashing, token blocklist in Redis

STEP 1 OF 2 Auth Implementation Prompt

```
Implement the complete Authentication module for the PDF Manager app.
All code goes in backend/app/. Use the logging infrastructure from Module 01.

1. SECURITY UTILITIES - app/core/security.py:
   - hash_password(plain: str) -> str  using passlib bcrypt
   - verify_password(plain: str, hashed: str) -> bool
   - create_access_token(data: dict) -> str  (HS256 JWT, 15 min expiry)
   - create_refresh_token(data: dict) -> str  (HS256 JWT, 7 day expiry)
   - decode_token(token: str) -> dict  (raises HTTPException 401 on invalid)
   - revoke_refresh_token(jti: str, redis_client) -> None
     (adds jti to Redis SET 'revoked_tokens' with 7-day TTL)
   - is_token_revoked(jti: str, redis_client) -> bool

2. PYDANTIC SCHEMAS - app/schemas/auth.py:
   SignupRequest: email(EmailStr), display_name(str, 2-100 chars), password(str,
8+ chars)
   LoginRequest: email, password
   TokenResponse: access_token, refresh_token, token_type='bearer'
   UserResponse: id, email, display_name, created_at

3. USER SERVICE - app/services/user_service.py:
   create_user(db, req: SignupRequest) -> User
     - Check email uniqueness, raise HTTP 409 if taken
     - Hash password, insert user, commit
     - log_event(logger, 'SIGNUP_SUCCESS', user_id=..., metadata={email})
   authenticate_user(db, email, password) -> User
     - Fetch user by email; log LOGIN_USER_NOT_FOUND and raise 401 if missing
     - Verify password; log LOGIN_BAD_PASSWORD and raise 401 if wrong
     - log_event(logger, 'LOGIN_SUCCESS', user_id=...)

4. AUTH ROUTER - app/api/auth.py  (prefix /api/auth):
   POST /signup:
     - log SIGNUP_ATTEMPT at start
     - call create_user(); return UserResponse + 201
   POST /login:
     - log LOGIN_ATTEMPT at start
     - authenticate_user(); create both tokens; return TokenResponse
   POST /logout:
     - Verify access token; extract refresh jti from body
     - revoke_refresh_token(); log LOGOUT + TOKEN_REVOKED
     - return {message: 'logged out'}
   POST /refresh:
     - Accept {refresh_token} body
```

```
      - Decode, check revocation, issue new access token
      - log TOKEN_REFRESHED

5. AUTH DEPENDENCY - app/core/deps.py:
   get_current_user(token: str = Depends(oauth2_scheme), db=...) -> User
      - Decode JWT, fetch user from DB, return User ORM object
      - Raise HTTP 401 if anything fails

6. RATE LIMITING:
   Apply SlowAPI limiter: 5 req/min on /signup and /login per IP.

Every log call must include request_id from the ContextVar middleware.
```

STEP 2 OF 2 Auth E2E Tests Prompt

```
Write and run E2E tests for the Authentication module.
File: backend/tests/e2e/test_01_auth.py
Use pytest + httpx AsyncClient against the running FastAPI app.

IMPORTANT: Each test is independent. Use a unique email per test (uuid suffix).
Clean up created users after each test using direct DB access.

TEST CASES:

test_signup_success:
  POST /api/auth/signup with valid data
  Assert: 201, response has id + email + display_name
  Assert: user row exists in DB with hashed (not plain) password
  Assert: logs table has a SIGNUP_SUCCESS row for this user

test_signup_duplicate_email:
  POST /api/auth/signup twice with same email
  Assert: second request returns 409
  Assert: only one user row in DB for that email

test_signup_weak_password:
  POST /api/auth/signup with 5-char password
  Assert: 422 Unprocessable Entity

test_login_success:
  Create user, then POST /api/auth/login
  Assert: 200, response has access_token + refresh_token
  Assert: both tokens are valid JWTs (decode without error)
  Assert: logs table has LOGIN_SUCCESS row

test_login_wrong_password:
  POST /api/auth/login with correct email, wrong password
  Assert: 401
  Assert: logs table has LOGIN_BAD_PASSWORD row
```

```
test_login_unknown_email:
  POST /api/auth/login with non-existent email
  Assert: 401
  Assert: logs table has LOGIN_USER_NOT_FOUND row

test_logout_invalidates_refresh_token:
  Login, then POST /api/auth/logout with refresh_token
  Then POST /api/auth/refresh with the same refresh_token
  Assert: refresh returns 401 (token revoked)
  Assert: Redis has the jti in revoked_tokens set

test_refresh_token_success:
  Login, POST /api/auth/refresh with valid refresh_token
  Assert: 200, new access_token returned
  Assert: new access_token is a different string from original

test_protected_route_without_token:
  GET /api/users/me with no Authorization header
  Assert: 401

test_protected_route_with_expired_token:
  Create a JWT with exp = now - 1 second
  GET /api/users/me with it
  Assert: 401

test_request_id_present_on_all_responses:
  POST /api/auth/signup, login, logout
  Assert: every response has X-Request-ID header (non-empty UUID)

SELF-HEALING RULE:
After running pytest, capture any FAILED tests.
For each failure: read the error, locate the bug in the source (not the test),
fix it, and re-run only the failed tests.
Repeat until all 11 tests pass.
Print final: AUTH MODULE — PASSED 11/11 tests.
```

MODULE 03

PDF Upload Module

Multi-file upload, async Celery processing, metadata extraction, status polling

STEP 1 OF 2 Upload Implementation Prompt

```
Implement the PDF Upload module. Assume Module 01 (infra) and Module 02 (auth) are
complete.

1. FILE STORAGE SERVICE - app/services/storage_service.py:
   - Base path from env FILE_STORAGE_PATH
   - save_upload(file_bytes: bytes, filename: str, user_id: UUID) -> str
     Write to {base}/{user_id}/{uuid}_{sanitized_filename}
     Use atomic write: write to .tmp path, rename on success
     Return relative storage_path
   - read_file(storage_path: str) -> AsyncGenerator[bytes]
     Async byte-range streaming generator
   - delete_file(storage_path: str) -> None

2. CELERY TASK - app/tasks/pdf_tasks.py:
   Task: process_uploaded_pdf(pdf_id: str, storage_path: str, user_id: str)
   - log TASK_STARTED with task_id, pdf_id
   - Open file with pikepdf, extract page_count
   - Update pdfs row: page_count, status='ready', updated_at=now
   - log TASK_COMPLETED with duration_ms
   - On any exception: update status='error', log TASK_FAILED with error traceback
   - Retry up to 3 times with exponential backoff (countdown=2**retries)
   Celery signal handlers:
   - task_prerun: log TASK_PRERUN with task name, task_id, args
   - task_postrun: log TASK_POSTRUN with state, duration_ms

3. UPLOAD ROUTER - app/api/pdfs.py  (prefix /api/pdfs):
   POST /upload (multipart, auth required):
     - Accept List[UploadFile] (field name: files)
     - For each file:
       a. Validate content_type == 'application/pdf'
          Also validate first 4 bytes == b'%PDF' (magic byte check)
          Validate size <= MAX_UPLOAD_SIZE_MB * 1024 * 1024
          Return 422 immediately for any invalid file
       b. Read bytes, call storage_service.save_upload()
       c. Insert pdfs row with status='pending'
       d. Enqueue process_uploaded_pdf.delay(pdf_id, storage_path, user_id)
       e. log UPLOAD_RECEIVED and TASK_ENQUEUED with file details
     - Return 202 with list of {pdf_id, filename, status: 'pending', task_id}
     - Total endpoint must return within 500ms regardless of file size

   GET /tasks/{task_id}  (auth required):
     - Query Celery AsyncResult for the task_id
     - Return {task_id, state, result, progress}
     - Log TASK_STATUS_POLLED
```

```
4. RATE LIMITING:
   10 uploads per minute per user on POST /api/pdfs/upload

5. CELERY APP CONFIG - app/tasks/__init__.py:
   broker_url = REDIS_URL
   result_backend = REDIS_URL
   task_serializer = 'json'
   task_acks_late = True  (reliability)
   worker_prefetch_multiplier = 1
```

STEP 2 OF 2 Upload E2E Tests Prompt

```
Write and run E2E tests for the PDF Upload module.
File: backend/tests/e2e/test_02_upload.py

SETUP: create a conftest.py fixture that logs in a test user and provides
auth_headers. Create a small valid PDF fixture (3 pages, ~50KB) using pikepdf.
Create a helper wait_for_status(pdf_id, target_status, timeout=30) that polls
GET /api/pdfs/{id} every second until status matches or raises TimeoutError.

TEST CASES:

test_single_pdf_upload_returns_202:
  POST /api/pdfs/upload with 1 valid PDF
  Assert: 202 response within 500ms (use time.perf_counter)
  Assert: response body has pdf_id, filename, status='pending', task_id
  Assert: pdfs row in DB with status='pending'

test_processing_completes_status_ready:
  Upload 1 PDF, capture pdf_id
  Poll wait_for_status(pdf_id, 'ready', timeout=30)
  Assert: pdfs row has status='ready' and page_count > 0
  Assert: logs table has TASK_COMPLETED row for this pdf_id

test_multi_file_upload:
  POST /api/pdfs/upload with 3 PDFs in one request
  Assert: 202, response list has 3 items each with unique pdf_id
  Wait for all 3 to reach status='ready'
  Assert: 3 separate pdfs rows in DB owned by test user

test_invalid_file_type_rejected:
  POST /api/pdfs/upload with a .txt file (text/plain)
  Assert: 422
  Assert: no pdfs row created in DB

test_fake_pdf_magic_bytes_rejected:
  POST /api/pdfs/upload with a file named 'test.pdf' but content is 'NOT A PDF'
  Assert: 422
  Assert: no pdfs row created, no Celery task enqueued
```

```
test_oversized_file_rejected:
  Create a bytes object of size (MAX_UPLOAD_SIZE_MB + 1) * 1024 * 1024
  POST /api/pdfs/upload with it
  Assert: 422

test_upload_requires_auth:
  POST /api/pdfs/upload with no auth headers
  Assert: 401

test_task_status_polling:
  Upload a PDF, capture task_id
  GET /api/pdfs/tasks/{task_id}
  Assert: 200, body has task_id + state field

test_upload_logs_emitted:
  Upload 1 PDF, wait for 'ready'
  Query logs table for this pdf's events
  Assert: logs contain all of: UPLOAD_RECEIVED, TASK_ENQUEUED,
          TASK_STARTED, TASK_COMPLETED (in that order by timestamp)

test_atomic_write_no_partial_files:
  Simulate a write failure by temporarily making the storage dir read-only
  POST /api/pdfs/upload
  Assert: no partial .tmp file left behind in storage dir
  (Restore permissions before next test)

SELF-HEALING RULE:
Run all tests. For each failure, read the traceback, find the bug in source,
fix it, re-run the failed test. Repeat until all 10 tests pass.
Print: UPLOAD MODULE — PASSED 10/10 tests.
```

MODULE 04

PDF Viewing & File Management

Dashboard listing, streaming viewer, rename, soft-delete with 30-day purge

STEP 1 OF 2 Viewing & Management Implementation Prompt

```
Implement PDF listing, streaming viewer, rename, and delete endpoints.
Assume Modules 01-03 are complete.

1. PDF SCHEMAS - app/schemas/pdf.py:
   PDFResponse: id, name, size_bytes, page_count, status, version,
                created_at, updated_at
   PDFListResponse: items: List[PDFResponse], total, page, size
   RenameRequest: name (str, 1-255 chars, must end with .pdf)

2. PDF SERVICE - app/services/pdf_service.py:
   list_pdfs(db, user_id, page=1, size=20) -> PDFListResponse
     SELECT WHERE owner_id=user_id AND deleted_at IS NULL ORDER BY created_at DESC
   get_pdf(db, pdf_id, user_id) -> PDF  (raises 404/403 if missing/wrong owner)
   rename_pdf(db, pdf_id, user_id, new_name) -> PDF
     Validate name, update row, log RENAME_SUCCESS with old_name -> new_name
   soft_delete_pdf(db, pdf_id, user_id) -> None
     Set deleted_at=now(), log SOFT_DELETE_SUCCESS with pdf_id

3. PDF ROUTER - additional endpoints in app/api/pdfs.py:

   GET / (auth required, paginated):
     Query params: page(default=1), size(default=20, max=100)
     Return PDFListResponse
     log PDF_LIST_FETCHED with page, size, total

   GET /{id} (auth required):
     Return PDFResponse for one file
     Verify owner, raise 403 if mismatch
     log PDF_DETAIL_FETCHED

   GET /{id}/stream (auth required):
     - Verify ownership
     - log VIEW_REQUEST_INITIATED
     - Stream file bytes using StreamingResponse + storage_service.read_file()
     - Set headers: Content-Type: application/pdf,
       Content-Disposition: inline; filename=...,
       Accept-Ranges: bytes
     - Support HTTP Range header for partial content (byte-range streaming)
     - log FILE_STREAM_COMPLETE with bytes_transferred, duration_ms

   PATCH /{id} (auth required):
     Body: RenameRequest
     Call rename_pdf(); return updated PDFResponse
```

```
   DELETE /{id} (auth required):
     Call soft_delete_pdf()
     Return 204 No Content

4. PERIODIC TASK - add to app/tasks/pdf_tasks.py:
   purge_deleted_pdfs() — Celery beat task, runs daily at 02:00 UTC:
     SELECT pdfs WHERE deleted_at < now() - interval '30 days'
     For each: call storage_service.delete_file(), hard-delete DB row
     log PURGE_COMPLETED with count of files purged

5. CELERY BEAT SCHEDULE - add to Celery app config:
   'purge-deleted-pdfs': {'task': 'purge_deleted_pdfs', 'schedule':
crontab(hour=2, minute=0)}
```

STEP 2 OF 2 Viewing & Management E2E Tests Prompt

```
Write and run E2E tests for PDF Viewing and File Management.
File: backend/tests/e2e/test_03_view_manage.py

SETUP: fixture that uploads 3 PDFs and waits for all to reach status='ready'.

TEST CASES:

test_list_pdfs_authenticated:
  GET /api/pdfs (with auth)
  Assert: 200, body has items array + total + page + size
  Assert: all returned items have owner_id == test user

test_list_pdfs_pagination:
  Upload 5 PDFs, wait for ready
  GET /api/pdfs?page=1&size=2
  Assert: items has exactly 2 entries, total >= 5
  GET /api/pdfs?page=2&size=2
  Assert: items has up to 2 different entries

test_get_single_pdf:
  GET /api/pdfs/{id} for an uploaded PDF
  Assert: 200, id + name + status='ready' + page_count > 0

test_get_pdf_wrong_owner:
  Create a second user, upload a PDF as that user
  Try GET /api/pdfs/{that_pdf_id} as first user
  Assert: 403

test_stream_pdf:
  GET /api/pdfs/{id}/stream
  Assert: 200, Content-Type: application/pdf
  Assert: first 4 bytes of response body == b'%PDF'
  Assert: response time < 800ms for test PDF
```

```
test_stream_pdf_range_request:
  GET /api/pdfs/{id}/stream with header Range: bytes=0-1023
  Assert: 206 Partial Content
  Assert: Content-Range header present
  Assert: body length == 1024 bytes

test_stream_requires_auth:
  GET /api/pdfs/{id}/stream with no auth
  Assert: 401

test_rename_pdf:
  PATCH /api/pdfs/{id} with {name: 'new-name.pdf'}
  Assert: 200, response name == 'new-name.pdf'
  Assert: pdfs row in DB updated
  Assert: logs table has RENAME_SUCCESS with old_name + new_name

test_rename_without_pdf_extension:
  PATCH /api/pdfs/{id} with {name: 'no-extension'}
  Assert: 422

test_rename_empty_name:
  PATCH /api/pdfs/{id} with {name: ''}
  Assert: 422

test_soft_delete:
  DELETE /api/pdfs/{id}
  Assert: 204
  Assert: pdfs row has deleted_at set (not null)
  Assert: pdfs row still exists in DB (soft delete only)
  Assert: GET /api/pdfs/{id} returns 404 (excluded from queries)
  Assert: logs table has SOFT_DELETE_SUCCESS row

test_deleted_pdf_not_in_list:
  Upload 2 PDFs, delete 1
  GET /api/pdfs
  Assert: deleted PDF does not appear in items
  Assert: other PDF still appears

test_view_logs_emitted:
  Stream a PDF
  Assert: logs table has VIEW_REQUEST_INITIATED and FILE_STREAM_COMPLETE rows

SELF-HEALING RULE:
Run all tests. Fix any failures in source code, re-run until all 13 tests pass.
Print: VIEW/MANAGE MODULE — PASSED 13/13 tests.
```

MODULE 05

PDF Editing Module

Server-side PDF editing with pikepdf: insert text, erase regions, save versions

STEP 1 OF 2 PDF Editing Implementation Prompt

```
Implement server-side PDF editing using pikepdf. Modules 01-04 are complete.

ARCHITECTURE DECISION: Editing is handled server-side via pikepdf.
The React frontend will send structured edit operations as JSON; the backend
applies them and returns the updated PDF. This is more reliable than
client-side editing for complex PDFs.

1. EDIT OPERATION SCHEMAS - app/schemas/pdf.py (add to existing):
   TextInsertOp:
     type: Literal['insert_text']
     page: int (0-indexed)
     x: float, y: float  (points from bottom-left)
     text: str
     font_size: float (default 12)
     color_hex: str (default '#000000')

   EraseOp:
     type: Literal['erase']
     page: int
     x: float, y: float, width: float, height: float  (bounding rect in points)

   EditRequest:
     operations: List[Union[TextInsertOp, EraseOp]]  (max 100 ops)
     comment: Optional[str]  (describes what was changed)

   EditResponse:
     pdf_id: UUID, version: int, saved_at: datetime

2. EDIT SERVICE - app/services/edit_service.py:
   apply_operations(source_path: str, ops: List[op], dest_path: str) -> int:
     Opens PDF with pikepdf
     For each operation:
       insert_text: create a content stream overlay on the target page
         using pikepdf's PDF content stream API to stamp text at (x,y)
         with specified font size and RGB color
       erase: stamp a white filled rectangle over the bounding box
         using pikepdf content stream (covers existing content)
     Save to dest_path (atomic: write tmp, rename)
     Return new page count

3. CELERY TASK - add to app/tasks/pdf_tasks.py:
   save_edited_pdf(pdf_id, user_id, operations_json, comment):
     - log EDIT_TASK_STARTED
```

```
      - Load current storage_path from DB
      - Archive current file as pdf_versions row (version = current version)
      - Generate new storage path (same dir, new uuid filename)
      - Call edit_service.apply_operations(old_path, ops, new_path)
      - Update pdfs row: storage_path=new_path, version+=1, updated_at=now
      - log EDIT_TASK_COMPLETED with version, ops_count, duration_ms
      - On error: log EDIT_TASK_FAILED, do NOT update DB (original preserved)

4. EDIT ENDPOINTS - add to app/api/pdfs.py:
   POST /{id}/edit  (auth required):
      - Verify ownership
      - Validate EditRequest (max 100 ops)
      - Enqueue save_edited_pdf task
      - log EDIT_REQUEST_RECEIVED with op count, comment
      - Return 202 with task_id and current version

   GET /{id}/versions  (auth required):
      - Return list of pdf_versions rows for this pdf
      - Each: version number, saved_at, storage_path (omit direct path, use version
id)
      - log VERSION_HISTORY_FETCHED

   GET /{id}/versions/{version_id}/stream  (auth required):
      - Stream a specific historical version's bytes
      - Same streaming logic as main stream endpoint
      - log VERSION_STREAM_FETCHED
```

STEP 2 OF 2 PDF Editing E2E Tests Prompt

```
Write and run E2E tests for the PDF Editing module.
File: backend/tests/e2e/test_04_edit.py

SETUP: fixture uploads a known 3-page PDF and waits for status='ready'.
Helper: wait_for_version(pdf_id, expected_version, timeout=30) polls
GET /api/pdfs/{id} until version matches.

TEST CASES:

test_insert_text_operation:
  POST /api/pdfs/{id}/edit with one TextInsertOp on page 0 at (72, 700)
    text='Hello World', font_size=14, color_hex='#FF0000'
  Assert: 202, body has task_id
  Wait for version to increment to 2
  GET /api/pdfs/{id} — Assert version == 2
  GET /api/pdfs/{id}/stream — download new PDF
  Open with pikepdf locally, extract page 0 content stream
  Assert: text 'Hello World' appears in content stream bytes

test_erase_operation:
  POST /api/pdfs/{id}/edit with one EraseOp on page 0
    x=72, y=700, width=200, height=30
```

```
  Assert: 202
  Wait for version increment
  Download PDF, verify it has same page count
  Assert: white rectangle element present in page 0 content stream

test_multiple_operations_in_one_request:
  POST /api/pdfs/{id}/edit with 3 ops: 2 text inserts + 1 erase
  Assert: 202
  Wait for version increment
  Assert: version == previous + 1 (all ops applied in one save)

test_version_history_recorded:
  Apply 2 edit operations (separate requests, wait for each)
  GET /api/pdfs/{id}/versions
  Assert: 200, list has at least 2 entries
  Assert: each has version number and saved_at timestamp
  Assert: versions are ordered (ascending version number)

test_stream_historical_version:
  Apply an edit, get version list
  GET /api/pdfs/{id}/versions/{old_version_id}/stream
  Assert: 200, Content-Type: application/pdf
  Assert: streamed bytes are original (pre-edit) PDF
  Verify by checking page content does NOT contain inserted text

test_edit_preserves_original_on_failure:
  Inject a bad operation (invalid page number = 9999)
  POST /api/pdfs/{id}/edit
  Wait for task to fail (poll task status until state='FAILURE')
  GET /api/pdfs/{id} — Assert version unchanged
  GET /api/pdfs/{id}/stream — Assert original bytes still intact (starts with
%PDF)

test_max_operations_limit:
  POST /api/pdfs/{id}/edit with 101 operations
  Assert: 422 (exceeds max 100 ops)

test_edit_requires_ownership:
  Create second user, try to edit first user's PDF
  Assert: 403

test_edit_logs_emitted:
  Submit an edit, wait for completion
  Query logs table for pdf_id
  Assert: EDIT_REQUEST_RECEIVED, EDIT_TASK_STARTED, EDIT_TASK_COMPLETED
           all present in correct chronological order

SELF-HEALING RULE:
Run all tests. Fix any failures in source. Re-run until all 9 tests pass.
Print: EDIT MODULE — PASSED 9/9 tests.
```

MODULE 06

User Profile Module

View profile, update display name, change email with password confirmation, change password

STEP 1 OF 2 User Profile Implementation Prompt

```
Implement the User Profile module. Modules 01-05 are complete.

1. USER SCHEMAS - app/schemas/user.py:
   UserResponse: id, email, display_name, created_at, updated_at
   UpdateProfileRequest:
     display_name: Optional[str] (2-100 chars)
     email: Optional[EmailStr]
     current_password: Optional[str]  (required when changing email or password)
     new_password: Optional[str] (8+ chars, 1 uppercase, 1 digit)
   VALIDATION RULES (pydantic validators):
     - If email is provided, current_password must also be provided
     - If new_password is provided, current_password must also be provided
     - new_password must differ from current_password

2. USER SERVICE - app/services/user_service.py (extend existing):
   get_user_by_id(db, user_id) -> User

   update_profile(db, user_id, req: UpdateProfileRequest) -> User:
     a. Fetch current user
     b. If display_name provided: update field, log DISPLAY_NAME_CHANGED
     c. If email or new_password provided:
        - Verify current_password; raise 401 if wrong, log WRONG_CURRENT_PASSWORD
        - If email: check uniqueness (409 if taken), update, log EMAIL_CHANGED
        - If new_password: hash + update, log PASSWORD_CHANGED
          (NEVER log the password value — only log 'field: password')
     d. Set updated_at = now(), commit, return refreshed user

3. USER ROUTER - app/api/users.py  (prefix /api/users):
   GET /me  (auth required):
     Return UserResponse for current user
     log PROFILE_FETCHED

   PATCH /me  (auth required):
     Accept UpdateProfileRequest
     Call update_profile()
     Return updated UserResponse
     log PROFILE_UPDATE_REQUEST at start, log individual field changes

4. INPUT SANITISATION:
   Trim whitespace from display_name and email before saving.
   Reject display_name with only whitespace (422).
```

```
5. PASSWORD RULES VALIDATOR:
   Create a reusable Pydantic validator: validate_password_strength(v: str):
     - min 8 chars
     - at least 1 uppercase letter
     - at least 1 digit
     Raise ValueError with descriptive message if any rule fails.
   Apply this validator in both SignupRequest and UpdateProfileRequest.
```

STEP 2 OF 2 User Profile E2E Tests Prompt

```
Write and run E2E tests for the User Profile module.
File: backend/tests/e2e/test_05_profile.py

SETUP: fixture that creates a fresh user and returns auth_headers + user_id.

TEST CASES:

test_get_my_profile:
  GET /api/users/me with valid auth
  Assert: 200, body has id, email, display_name, created_at
  Assert: email matches registered email

test_get_profile_requires_auth:
  GET /api/users/me with no token
  Assert: 401

test_update_display_name:
  PATCH /api/users/me with {display_name: 'New Name'}
  Assert: 200, response display_name == 'New Name'
  GET /api/users/me — Assert: display_name persisted
  Assert: logs has DISPLAY_NAME_CHANGED row for this user

test_update_display_name_whitespace_only:
  PATCH /api/users/me with {display_name: '   '}
  Assert: 422

test_update_display_name_too_short:
  PATCH /api/users/me with {display_name: 'A'}
  Assert: 422

test_change_email_with_correct_password:
  PATCH /api/users/me with {email: 'new@example.com', current_password: '...'}
  Assert: 200, response email == 'new@example.com'
  Try to login with new email — Assert: 200
  Try to login with old email — Assert: 401
  Assert: logs has EMAIL_CHANGED row

test_change_email_without_password:
  PATCH /api/users/me with {email: 'new@example.com'} (no current_password)
  Assert: 422
```

```
test_change_email_wrong_password:
  PATCH /api/users/me with {email: 'new@example.com', current_password: 'WRONG'}
  Assert: 401
  Assert: email NOT changed in DB

test_change_email_already_taken:
  Create second user with email B
  Try to update first user's email to email B
  Assert: 409

test_change_password:
  PATCH /api/users/me with {current_password: 'old', new_password: 'NewPass1'}
  Assert: 200
  Login with new password — Assert: 200
  Login with old password — Assert: 401
  Assert: logs has PASSWORD_CHANGED row
  Assert: logs row does NOT contain old or new password string

test_change_password_weak_new_password:
  PATCH /api/users/me with {current_password: 'old', new_password: 'weak'}
  Assert: 422

test_change_password_same_as_current:
  PATCH /api/users/me with same password for both current and new
  Assert: 422

test_no_op_update_does_not_fail:
  PATCH /api/users/me with {} (empty body)
  Assert: 200, user unchanged

SELF-HEALING RULE:
Run all tests. Fix any failures in source. Re-run until all 13 tests pass.
Print: USER PROFILE MODULE — PASSED 13/13 tests.
```

MODULE 07

Live Log Viewer Module

SSE real-time log stream to browser, filterable log history API, React log terminal

STEP 1 OF 2 Live Logs Implementation Prompt

```
Implement the Live Log Viewer — both backend SSE stream and React terminal UI.
Modules 01-06 are complete. This is the observability layer.

BACKEND:

1. LOG QUERY SERVICE - app/services/log_service.py:
   get_logs(db, user_id=None, level=None, module=None, page=1, size=50)
     SELECT from logs WHERE filters ORDER BY timestamp DESC LIMIT/OFFSET
     Note: user_id filter is optional (admin can see all; for personal app,
     you can scope to current user or show all)
   Returns: {items: List[LogEntry], total, page, size}

2. SSE STREAM ENDPOINT - app/api/logs.py  (prefix /api/logs):
   GET /stream  (auth required, SSE):
     - Return StreamingResponse with media_type='text/event-stream'
     - Generator function async_log_generator():
       * Connect to PostgreSQL LISTEN 'new_log_entry'
       * On each NOTIFY: parse payload JSON, yield 'data: {json}\n\n'
       * Send a keepalive comment (': ping\n\n') every 15 seconds
       * On client disconnect (GeneratorExit): UNLISTEN, close pg connection
     - Require auth (validate JWT from query param ?token= since SSE can't
       easily set headers in EventSource)

3. UPDATE DatabaseLogHandler to NOTIFY:
   After inserting a log row, execute:
     NOTIFY new_log_entry, '{json_payload}'
   where json_payload is the log entry serialised as JSON.
   This is done in the background thread — MUST use a separate sync pg connection
   (not the async SQLAlchemy session which is on the event loop thread).

4. LOG HISTORY ENDPOINT:
   GET /api/logs  (auth required):
     Query params: level, module, page, size
     Return paginated log history
     log LOGS_FETCHED (at DEBUG level to avoid infinite recursion)

FRONTEND:

5. React Log Terminal - src/pages/LogsPage.tsx:
   - Connect to /api/logs/stream via EventSource (pass token as query param)
   - Render entries in a dark-background scrollable div (monospace font)
   - Each entry: [timestamp] [LEVEL badge] [module] event | key=value pairs from
metadata
```

```
   - Color-code by level: DEBUG=gray, INFO=blue, WARNING=yellow, ERROR=red,
CRITICAL=red+bold
   - Controls:
     * Filter by level (dropdown, multi-select)
     * Filter by module (dropdown, multi-select)
     * Auto-scroll toggle (checkbox, default ON)
     * Clear display button (does not delete from DB, only clears the view)
     * Pause/Resume stream button
   - Show connection status badge: Connected (green) / Disconnected (red) /
Reconnecting
   - On disconnect: auto-reconnect with exponential backoff (1s, 2s, 4s, max 30s)
   - Show last 500 entries max in DOM to avoid memory issues (ring buffer)

6. Log Entry Component - src/components/LogEntry.tsx:
   Props: timestamp, level, module, event, user_id, request_id, duration_ms,
metadata
   Render as a single line with expandable metadata section on click
   Expanded view shows full JSON of the metadata field
```

STEP 2 OF 2 Live Logs E2E Tests Prompt

```
Write and run E2E tests for the Live Log Viewer module.
File: backend/tests/e2e/test_06_logs.py

NOTE: SSE streaming tests use httpx with a streaming context.

TEST CASES:

test_log_history_endpoint:
  Perform a login (generates log entries)
  GET /api/logs?page=1&size=10
  Assert: 200, items array present, each item has timestamp + level + event

test_log_history_filter_by_level:
  GET /api/logs?level=ERROR
  Assert: all returned items have level == 'ERROR'

test_log_history_filter_by_module:
  GET /api/logs?module=auth
  Assert: all returned items have module == 'auth'

test_log_history_pagination:
  Ensure at least 15 log entries exist (perform several actions)
  GET /api/logs?page=1&size=5
  Assert: items length == 5
  GET /api/logs?page=2&size=5
  Assert: items are different from page 1

test_log_history_requires_auth:
  GET /api/logs with no token
  Assert: 401
```

```
test_sse_stream_connects:
  Open SSE stream: GET /api/logs/stream?token={access_token}
  Collect events for 3 seconds
  Perform a login in parallel (generates a log entry)
  Assert: at least one SSE event received within 3 seconds
  Assert: event data is valid JSON with timestamp + level + event fields

test_sse_stream_receives_login_event:
  Open SSE stream in background task
  Perform POST /api/auth/login
  Assert: within 2 seconds, stream receives an event with event='LOGIN_SUCCESS'

test_sse_stream_receives_upload_event:
  Open SSE stream in background task
  Upload a PDF
  Assert: within 5 seconds, stream receives UPLOAD_RECEIVED event
  Assert: stream also receives TASK_ENQUEUED event

test_sse_keepalive_sent:
  Open SSE stream, wait 20 seconds
  Assert: at least one SSE comment line received (': ping')
  (Tests that connection stays alive without data)

test_sse_invalid_token_rejected:
  GET /api/logs/stream?token=invalid_jwt
  Assert: connection closed with 401 or error event

test_log_metadata_jsonb_queryable:
  Upload a PDF, wait for TASK_COMPLETED log
  Query DB directly: SELECT * FROM logs WHERE metadata->>'pdf_id' = '{pdf_id}'
  Assert: at least one row returned
  Assert: the JSONB metadata field contains pdf_id key

test_celery_task_logs_in_db:
  Upload a PDF, wait for processing
  Query logs table WHERE task_id IS NOT NULL
  Assert: TASK_STARTED and TASK_COMPLETED rows exist with same task_id
  Assert: TASK_COMPLETED has duration_ms > 0

SELF-HEALING RULE:
Run all tests. Fix any failures in source. Re-run until all 12 tests pass.
Print: LOGS MODULE — PASSED 12/12 tests.
```

MODULE 08

React Frontend — Full UI

All pages: Auth, Dashboard, Viewer, Editor, Profile, Log Terminal — wired to the API

STEP 1 OF 1 Frontend Implementation Prompt

```
Implement the complete React frontend. All backend modules 01-07 are complete.
Tech: React 18 + TypeScript + Vite + Tailwind CSS + React Router v6.

1. API CLIENT - src/api/client.ts:
   Axios instance with baseURL='/api'
   Request interceptor: attach Authorization: Bearer {access_token} from
   localStorage (or in-memory store — do NOT use cookies for token here,
   since the backend uses query-param auth for SSE).
   Response interceptor: on 401, call POST /auth/refresh, retry original request.
   On refresh failure: clear tokens, redirect to /login.

2. ROUTE STRUCTURE - src/App.tsx:
   / -> redirect to /dashboard if authenticated, /login if not
   /signup -> SignupPage
   /login -> LoginPage
   /dashboard -> DashboardPage (protected)
   /pdf/:id/view -> ViewerPage (protected)
   /pdf/:id/edit -> EditorPage (protected)
   /profile -> ProfilePage (protected)
   /logs -> LogsPage (protected)

3. AUTH PAGES:
   SignupPage: email, display_name, password fields
     On success: redirect to /login with success toast
   LoginPage: email, password
     On success: store tokens, redirect to /dashboard

4. DASHBOARD PAGE - src/pages/DashboardPage.tsx:
   - Top bar: app name, nav links (Dashboard, Logs, Profile), logout button
   - Upload zone: react-dropzone accepting .pdf only, shows file names selected
     Multi-file: show each file with a status badge (Pending / Processing / Ready
/ Error)
     Poll task status every 2 seconds per file until Ready
   - PDF grid: card per PDF — thumbnail (use first page preview if available,
     else a PDF icon), file name, size, page count, upload date, status badge
   - Each card has action buttons: View, Edit, Rename (inline edit), Delete
   - Rename: click to turn file name into an inline input; save on Enter or blur
   - Delete: show confirmation dialog (shadcn AlertDialog or custom modal)
   - Pagination controls: previous / next, current page / total indicator
   - Empty state: illustration + 'Upload your first PDF' message

5. VIEWER PAGE - src/pages/ViewerPage.tsx:
   - Use @react-pdf-viewer/core to render /api/pdfs/{id}/stream
   - Pass the Bearer token via a custom fetchOptions so the viewer can auth
```

```
   - Show toolbar: page number input, zoom controls, full-screen button,
     search box (built into @react-pdf-viewer)
   - 'Edit PDF' button in top-right -> navigate to EditorPage
   - Back to Dashboard link

6. EDITOR PAGE - src/pages/EditorPage.tsx:
   - Fetch PDF metadata, show file name + version badge
   - Operation builder panel (sidebar):
     * Add Text: page number, x, y, text, font_size, color picker
     * Erase Region: page number, x, y, width, height
     * List of pending operations (with remove button per op)
   - 'Save Changes' button: POST /api/pdfs/{id}/edit, show loading state,
     on success show toast 'Saved! Version {n}'
   - 'Version History' collapsible section: list versions with stream links
   - Note explaining coordinate system (0,0 = bottom-left of page in points)

7. PROFILE PAGE - src/pages/ProfilePage.tsx:
   Sections:
   - Display Info: show current name + email, edit button
   - Change Display Name: text input, save button
   - Change Email: new email + current password, save button
   - Change Password: current password + new password + confirm, save button
   Each section submits independently via PATCH /api/users/me.
   Show inline success/error messages per section.

8. LOGS PAGE - src/pages/LogsPage.tsx:
   (Full spec already in Module 07 Step 1 — implement exactly as described)

9. AUTH CONTEXT - src/context/AuthContext.tsx:
   Provides: user, tokens, login(), logout(), isAuthenticated
   ProtectedRoute component: redirect to /login if not authenticated

10. LOADING & ERROR STATES:
    - Every data-fetching component shows a skeleton loader while loading
    - All API errors show a toast (react-hot-toast) with the error message
    - 401 errors auto-trigger logout + redirect

NO CYPRESS/PLAYWRIGHT FOR THIS MODULE.
Instead: run the full Docker stack, manually verify each page renders and
core flows work end-to-end:
  [ ] Can sign up and log in
  [ ] Can upload 2 PDFs, both reach 'ready' status
  [ ] Can view a PDF in the viewer
  [ ] Can add a text operation and save (version increments)
  [ ] Can rename a PDF inline
  [ ] Can delete a PDF (with confirmation)
  [ ] Can change display name in profile
  [ ] Logs page shows real-time events for all of the above

Fix any runtime errors or broken flows before proceeding to Module 09.
```

MODULE 09

Integration & Final E2E Suite

Full cross-module E2E scenarios, CI smoke test script, coverage report

STEP 1 OF 1 Full Integration Test Suite Prompt

```
Write the final integration test suite covering end-to-end user journeys
across all modules. File: backend/tests/e2e/test_99_integration.py

These tests simulate real user behaviour from signup to final log observation.

JOURNEY 1 — Full User Lifecycle:
  1. POST /api/auth/signup — new user
  2. POST /api/auth/login — get tokens
  3. POST /api/pdfs/upload — 2 PDFs simultaneously
  4. Poll until both PDFs are status='ready'
  5. GET /api/pdfs — verify both appear in list
  6. GET /api/pdfs/{id}/stream — verify PDF bytes valid
  7. POST /api/pdfs/{id}/edit — insert text on page 0
  8. Poll until version == 2
  9. GET /api/pdfs/{id}/versions — verify 1 version entry
 10. PATCH /api/pdfs/{id} — rename to 'renamed.pdf'
 11. PATCH /api/users/me — update display_name
 12. DELETE /api/pdfs/{id} — soft delete first PDF
 13. GET /api/pdfs — verify only 1 PDF remains
 14. GET /api/logs?module=auth — verify login log present
 15. POST /api/auth/logout
  Assert: all 15 steps succeed, correct status codes throughout
  Assert: logs table has an entry for every major action above

JOURNEY 2 — Security Isolation Between Users:
  Create User A and User B (separate signups + logins)
  User A uploads a PDF
  User B attempts:
    GET /api/pdfs/{A's pdf_id}          — Assert: 403
    GET /api/pdfs/{A's pdf_id}/stream   — Assert: 403
    POST /api/pdfs/{A's pdf_id}/edit    — Assert: 403
    PATCH /api/pdfs/{A's pdf_id}        — Assert: 403
    DELETE /api/pdfs/{A's pdf_id}       — Assert: 403
  Assert: User A's PDF is completely unchanged after all B's attempts

JOURNEY 3 — Upload + Edit + Version Restore Flow:
  Upload PDF (version 1)
  Apply edit: insert 'V2 text' at page 0, x=72, y=700 (version 2)
  Apply edit: insert 'V3 text' at page 0, x=72, y=680 (version 3)
  GET /api/pdfs/{id}/versions — Assert: 2 version entries (v1 and v2 archived)
  Stream version 1 bytes — Assert: does NOT contain 'V2 text'
  Stream current (v3) bytes — Assert: contains both 'V2 text' and 'V3 text'
```

```
JOURNEY 4 — Log Completeness Audit:
  After running Journeys 1-3, query logs table
  Assert these event types all exist at least once:
    SIGNUP_SUCCESS, LOGIN_SUCCESS, LOGIN_BAD_PASSWORD,
    UPLOAD_RECEIVED, TASK_ENQUEUED, TASK_STARTED, TASK_COMPLETED,
    VIEW_REQUEST_INITIATED, FILE_STREAM_COMPLETE,
    EDIT_REQUEST_RECEIVED, EDIT_TASK_COMPLETED,
    RENAME_SUCCESS, SOFT_DELETE_SUCCESS,
    DISPLAY_NAME_CHANGED, LOGOUT, TOKEN_REVOKED
  Assert: no log row contains a raw password string
    (scan metadata JSONB and error fields for any of the test passwords)

JOURNEY 5 — Rate Limit Enforcement:
  Send 6 rapid POST /api/auth/login requests within 60 seconds
  Assert: at least 1 returns 429 Too Many Requests

COVERAGE REPORT:
  After all journey tests, run:
    pytest --cov=app --cov-report=term-missing tests/e2e/
  Assert: overall coverage >= 75%
  Print a coverage summary table (module: coverage %)

FINAL SUMMARY REPORT:
  Print a table of all E2E test files and their pass rates:
    test_00_infrastructure  : X/6
    test_01_auth            : X/11
    test_02_upload          : X/10
    test_03_view_manage     : X/13
    test_04_edit            : X/9
    test_05_profile         : X/13
    test_06_logs            : X/12
    test_99_integration     : X/5 journeys
    ___________________________
    TOTAL                   : X/79

SELF-HEALING RULE (final):
If any integration test fails, trace the failure to its root module,
fix the underlying source code, re-run the specific journey that failed,
then re-run the full suite to confirm no regressions.
Do not mark the project complete until TOTAL is 79/79.
```

Tips for Working with Cursor

Context Management

- Open each prompt in Cursor Composer (Cmd+I / Ctrl+I) for best results with large implementations.
- If Cursor loses context mid-generation, paste the prompt again with '@codebase' prefix.
- After each module, commit to git before starting the next one.

When Tests Fail

- Let Cursor's self-healing rule run — paste the failure output back if it doesn't pick it up automatically.
- For Celery task failures, check that the worker service is running: docker compose logs worker.
- For SSE/WebSocket test failures, increase timeout values — workers may be slower in test environments.

Environment

- Always run tests against docker compose services (not localhost directly) to match production config.
- Use docker compose exec api pytest tests/e2e/ to run tests inside the container.
- Add -v flag for verbose output: pytest -v tests/e2e/test_01_auth.py

Logging Debug Tip

- Tail live logs during tests: docker compose logs -f api worker
- The /logs page in the browser will show real-time events as tests execute — open it in parallel.

End of Cursor Prompts — PDF Manager Implementation Guide